



Introdução APIs RESTful



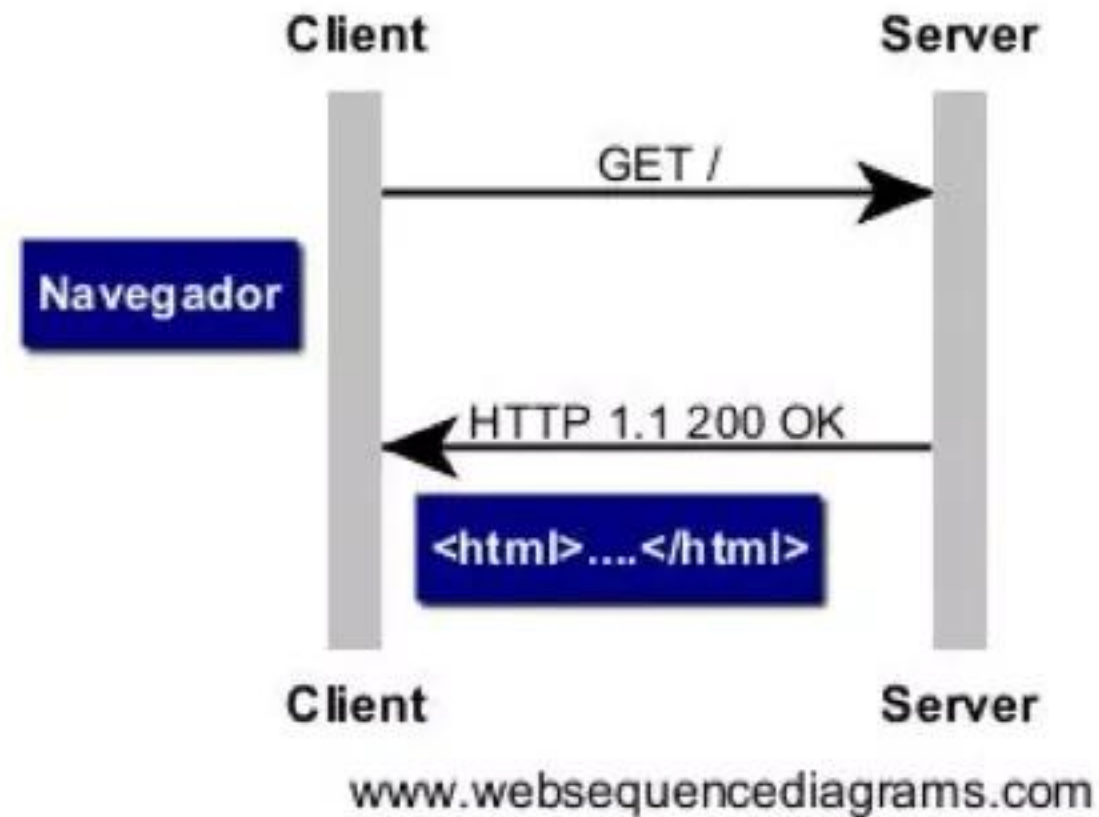


HTTP Basics



HTTP Características

- Método Request - Response
- URL (Uniform Resource Locator)
- Verbos GET, POST, PUT, etc..
- Permite Negociação de Dados
- Stateless



HTTP Requests

GET **/users/douglas** HTTP/1.1

Host: example.net

User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64; rv:44.0) Gecko/20100101 Firefox/44.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

HTTP Requests

GET /users/douglas HTTP/1.1

Host: example.net

User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64; rv:44.0) Gecko/20100101 Firefox/44.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

Verbs: GET, POST, PUT, DELETE,
PATCH

HTTP Requests

GET /users/douglas HTTP/1.1

Host: example.net

User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64; rv:44.0) Gecko/20100101 Firefox/44.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

URI: Uniform Resource Identifier

HTTP Requests

GET /users/douglas HTTP/1.1

Host: example.net

User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64; rv:44.0) Gecko/20100101 Firefox/44.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

Headers

Http Responses

HTTP/1.1 **200 OK**

Connection: close

Content-Type: text/html; charset=UTF-8

<html>

....

Http Responses

HTTP/1.1 **200 OK**

Connection: close

Content-Type: text/html; charset=UTF-8

<html>

....

Status Code “Comuns”

200 - OK	403 - Forbidden
201 - Created	404 - Not found
301 - Moved Permanently	410 - Gone
302 - Found	500 - Internal Server Error
304 - Not Modified	501 - Not Implemented
400 - Bad Request	502 - Bad Gateway
401 - Unauthorized	503 - Service Unavailable

https://pt.wikipedia.org/wiki/Lista_de_c%C3%B3digos_de_status_HTTP

Http Responses

HTTP/1.1 **200 OK**

Connection: close

Content-Type: text/html; charset=UTF-8

<html>

....

Http Responses

HTTP/1.1 **200 OK**

Connection: close

Content-Type: text/html; charset=UTF-8

<html>

....

BODY

Post - Exemplo

```
<form action="/usuarios/cadastrar.php" method="post">  
  Nome:<br />  
  <input type="text" name="nome"><br />  
  Sobrenome:<br />  
  <input type="text" name="sobrenome">  
  
  <input type="submit" value="cadastrar">  
</form>
```

Post - Exemplo

POST /usuarios/cadastrar.php HTTP/1.1

Host: example.net

User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64; rv:44.0) Gecko/20100101 Firefox/44.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

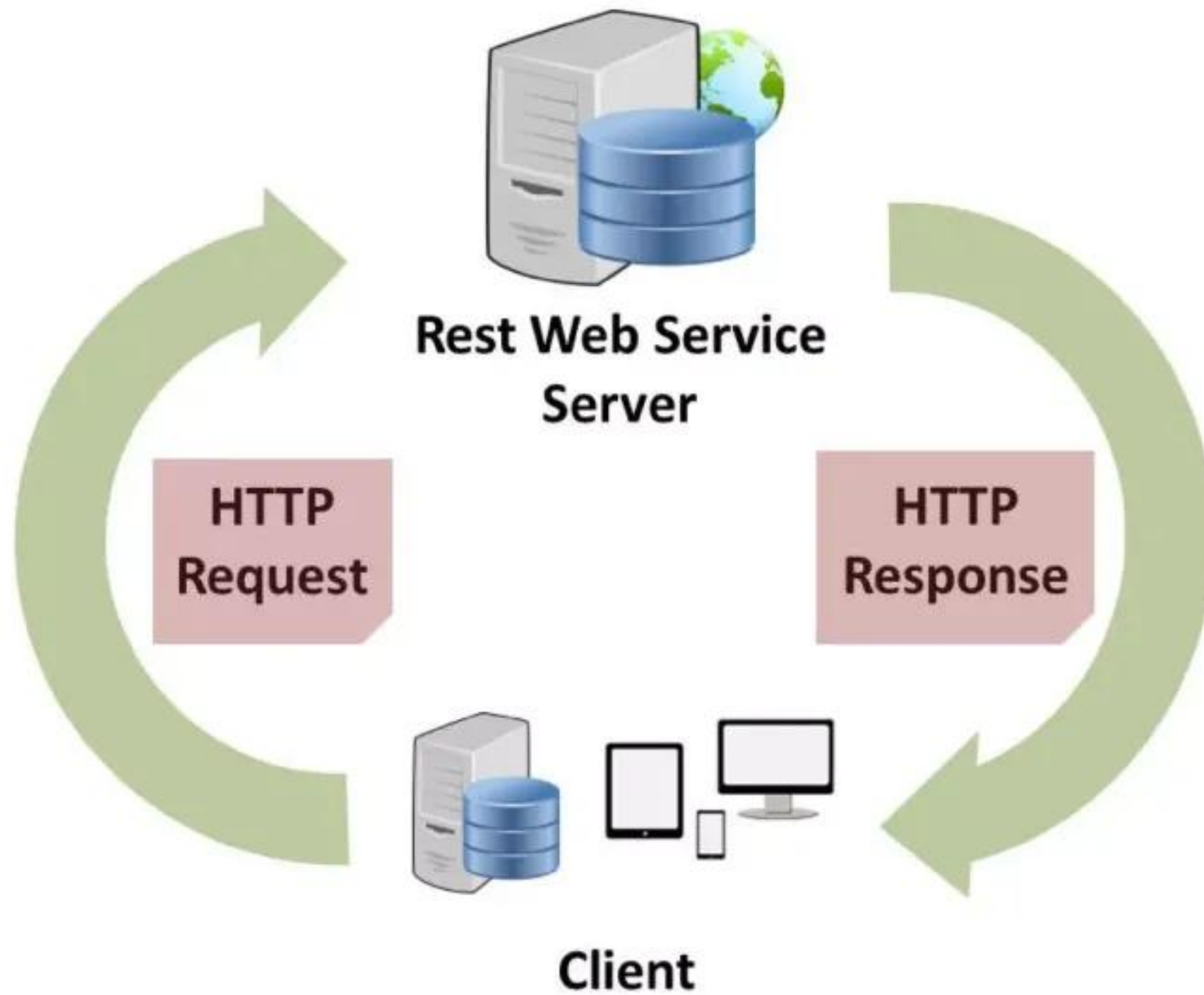
Content-Type: application/x-www-form-urlencoded

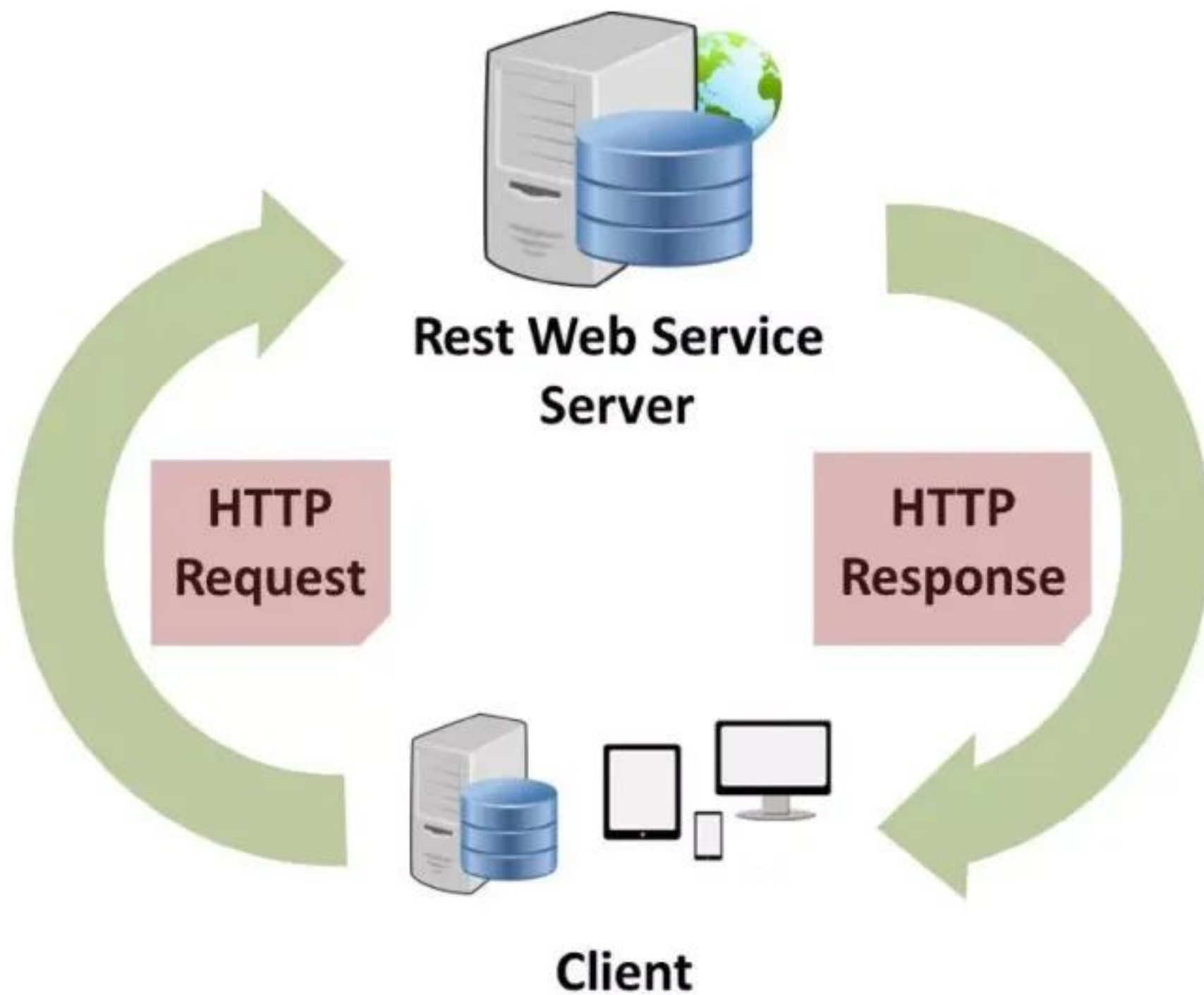
Content-Length: 32

nome=Douglas&sobrenome=V.+Pasqua



Webservices





Client

- Pcs
- Macs
- Mobile devices
- Outras aplicações
- etc..

Principais Serviços

- Remote Procedure Call (RPC)
- SOAP
- REST

REST é Popular



GitHub



YAHOO!
DEVELOPER
NETWORK



WORDPRESS



Dropbox

Webservices REST

Características REST

- Conjunto de princípios/idéias para criação de Webservices
- Não está amarrado exclusivamente ao HTTP
- Porém HTTP é amplamente usado e fornece enormes vantagens ao REST
- Uma **API REST** bem desenhada basicamente faz uso:
 - URIs Individuais por recurso.
 - **Verbos HTTP** (POST, PUT, GET e DELETE).
 - Uso apropriado de **Headers HTTP** e **Status code** de resposta de requisições.
 - Múltiplas opções de **formatos de representação** de recursos (**JSON, XML**, etc.)
 - Controles de **hipermídia**.

Vantagens REST

- Simplicidade
- Múltiplos tipos de formato de representação (JSON, XML, etc.)
- Parse de JSON é veloz
- JSON tem suporte nativo em Javascript
- JSON pode ser lido e escrito em qualquer linguagem de programação
- Cache
- Facilidade em escalabilidade

URIs Individuais por Recurso

<http://api.example.net/cantores/michaeljackson>

<http://api.example.net/albums/thisisit>

<http://api.example.net/cantores/>

<http://api.example.net/albums/>

<http://api.example.net/albums/michaeljackson>

Uso de Verbos HTTP - Criando (POST)

Requisição

POST /livros

Content-type: application/json

```
{  
  "titulo": "My New Book",  
  "autor": "Autor Foo",  
  "isbn": "111-11-1111-111-1"  
}
```


Uso de Verbos HTTP - Criando (POST)

Requisição

POST /livros

Content-type: application/json

```
{  
  "titulo": "My New Book",  
  "autor": "Autor Foo",  
  "isbn": "111-11-1111-111-1"  
}
```

Resposta

HTTP/1.1 201 Created

Host: localhost

Connection: close

Cache-Control: no-cache

Location: /livros/4

Uso de Verbos HTTP - Criando (POST)

Requisição

POST /livros

Content-type: application/json

```
{  
  "titulo": "My New Book",  
  "autor": "Autor Foo",  
  "isbn": "111-11-1111-111-1"  
}
```

Status Code

201 Created

400 Bad Request

406 Not acceptable

Resposta

HTTP/1.1 201 Created

Host: localhost

Connection: close

Cache-Control: no-cache

Location: /livros/4

Uso de Verbos HTTP - Consultando (GET)

Requisição

GET /livros/4

GET /livros

Uso de Verbos HTTP - Consultando (GET)

Requisição

```
GET /livros/4  
GET /livros
```

Resposta

```
HTTP/1.1 200 OK
```

```
Host: localhost
```

```
Connection: close
```

```
Content-type: application/json
```

```
{"titulo": "My New Book", "autor": "Autor Foo",  
  "isbn": "111-11-1111-111-1"}
```

Uso de Verbos HTTP - Consultando (GET)

Requisição

GET /livros/4
GET /livros

Resposta

HTTP/1.1 200 OK

Host: localhost

Connection: close

Content-type: application/json

```
{"titulo": "My New Book", "autor": "Autor Foo",  
  "isbn": "111-11-1111-111-1"}
```

Status Code

200 OK

404 Not Found

Uso de Verbos HTTP - Atualizando (PUT)

Requisição

PUT /livros/4

Content-type: application/json

```
{  
  "titulo": "My Book",  
  "autor": "Autor Qualquer",  
  "isbn": "111-11-1111-111-1"  
}
```


Uso de Verbos HTTP - Atualizando (PUT)

Requisição

PUT /livros/4

Content-type: application/json

```
{  
  "titulo": "My Book",  
  "autor": "Autor Qualquer",  
  "isbn": "111-11-1111-111-1"  
}
```

Resposta

HTTP/1.1 200 OK

Host: localhost

Connection: close

Cache-Control: no-cache

Content-type: application/json

```
{"titulo": "My Book", "autor": "Autor Qualquer",  
  "isbn": "111-11-1111-111-1"}
```


Uso de Verbos HTTP - Atualizando (PUT)

Requisição

PUT /livros/4

Content-type: application/json

```
{  
  "titulo": "My Book",  
  "autor": "Autor Qualquer",  
  "isbn": "111-11-1111-111-1"  
}
```

Status Code

200 OK

400 Bad Request

406 Not acceptable

Resposta

HTTP/1.1 200 OK

Host: localhost

Connection: close

Cache-Control: no-cache

Content-type: application/json

```
{"titulo": "My Book", "autor": "Autor Qualquer",  
  "isbn": "111-11-1111-111-1"}
```

Uso de Verbos HTTP - Atualizando (PATCH)

Requisição

PATCH /livros/4

Content-type: application/json

```
{  
  "autor": "Autor XPTO",  
}
```

Uso de Verbos HTTP - Atualizando (PATCH)

Requisição

```
PATCH /livros/4
Content-type: application/json
{
  "autor": "Autor XPTO",
}
```

Resposta

```
HTTP/1.1 200 OK
Host: localhost
Connection: close
Cache-Control: no-cache
Content-type: application/json

{"titulo": "My Book", "autor": "Autor XPTO",
 "isbn": "111-11-1111-111-1"}
```

Uso de Verbos HTTP - Atualizando (PATCH)

Requisição

```
PATCH /livros/4
Content-type: application/json
{
  "autor": "Autor XPTO",
}
```

Status Code

200 OK
400 Bad Request
406 Not acceptable

Resposta

```
HTTP/1.1 200 OK
Host: localhost
Connection: close
Cache-Control: no-cache
Content-type: application/json
```

```
{"titulo": "My Book", "autor": "Autor XPTO",  
  "isbn": "111-11-1111-111-1"}
```


Uso de Verbos HTTP - Atualizando (PATCH)

Requisição

```
PATCH /livros/4
Content-type: application/json
{
  "autor": "Autor XPTO",
}
```

Status Code

200 OK
400 Bad Request
406 Not acceptable

Resposta

```
HTTP/1.1 200 OK
Host: localhost
Connection: close
Cache-Control: no-cache
Content-type: application/json
```

```
{"titulo": "My Book", "autor": "Autor XPTO",  
  "isbn": "111-11-1111-111-1"}
```



Uso de Verbos HTTP - (Alternativa PATCH -> PUT)

Requisição

```
PUT /livros/4/autor
Content-type: application/json
{
  "autor": "Autor XPTO",
}
```

Status Code

200 OK
400 Bad Request
406 Not acceptable

Resposta

```
HTTP/1.1 200 OK
Host: localhost
Connection: close
Cache-Control: no-cache
Content-type: application/json
```

```
{"titulo": "My Book", "autor": "Autor XPTO",  
  "isbn": "111-11-1111-111-1"}
```

Uso de Verbos HTTP - Removendo (DELETE)

Requisição

DELETE /livros/4

Uso de Verbos HTTP - Removendo (DELETE)

Requisição

DELETE /livros/4

Resposta

HTTP/1.1 204 No Content

Host: localhost

Connection: close

Uso de Verbos HTTP - Removendo (DELETE)

Requisição

DELETE /livros/4

Resposta

HTTP/1.1 204 No Content

Host: localhost

Connection: close

Status Code

204 No Content

404 Not Found

Formatos de Representação

- JSON
- XML
- etc...

Formatos de Representação - Mime Type

JSON

application/json

XML

application/xml

Formatos de Representação - Resposta

JSON

Content-type: application/json

XML

Content-type: application/xml

Formatos de Representação - Requisitando

JSON

Accept: application/json

XML

Accept: application/xml

Be Stateless (sem estado)

- Cada requisição pode ser tratada isoladamente pelo webservice.
- Simplifica o design da aplicação. Não há necessidade de gerenciar interações prévias do usuário.
- Facilita escalabilidade.
- O Próprio protocolo HTTP é Stateless.

Controles de hipermídia

- Fornece “Links” para consumidores da API. Permitindo acessarem outros recursos relacionados. Como se fossem usuários clicando em links e botões na internet.

```
{  
  "titulo": "My Book",  
  "autor": "Autor XPTO",  
  "isbn": "111-11-1111-111-1",  
  "autorURL": "https://api.livros.example.net/autor/Autor XPTO",  
  "autorLivrosURL": "https://api.livros.example.net/autor/Autor XPTO/livros"  
}
```

Controles de hipermídia

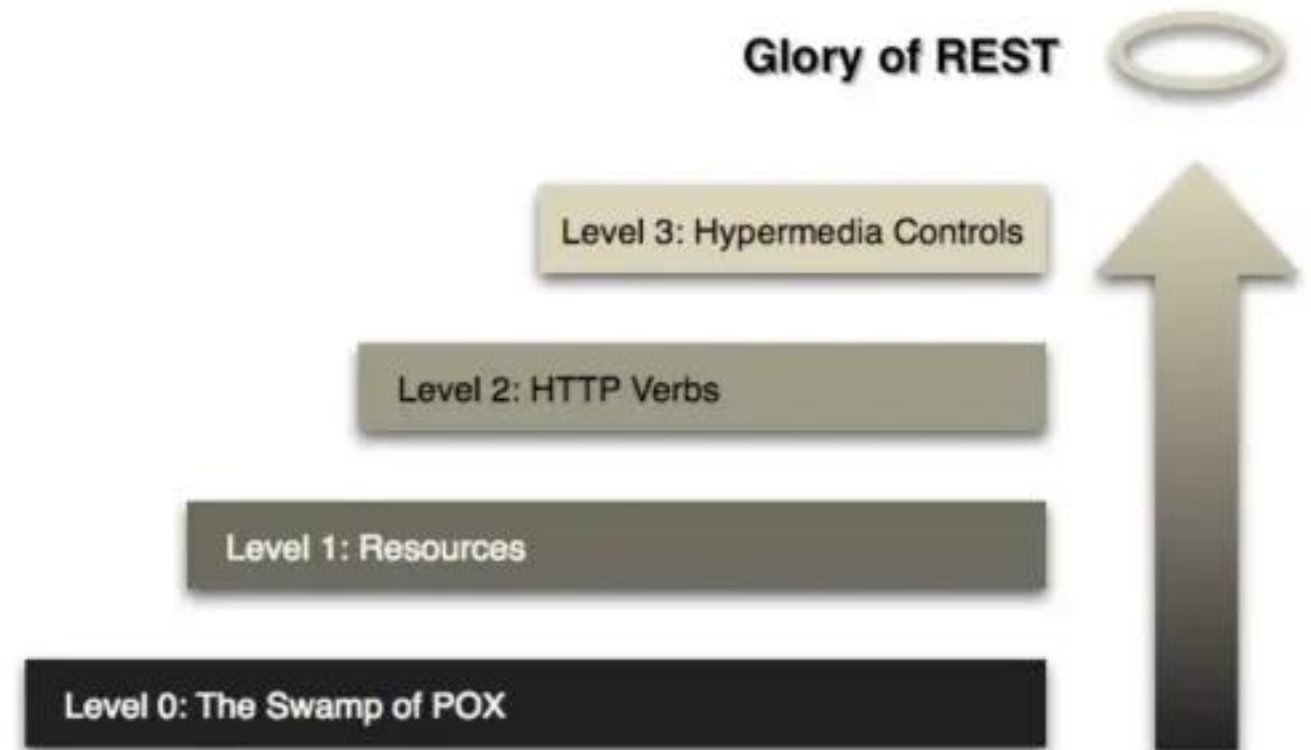
- HAL - Hypertext Application Language

application/hal+json

APIs com bom design - maturidade

- Richardson Maturity Model

- Level0: RPC básico, centralizado em único endpoint. (Sem uso de recursos HTTP)
- Level1: Expor recursos através de URIs individuais.
- Level2: Usar Verbos HTTP, Uso adequado dos códigos de status e Headers.
- Level3: Controler de hipermídia.



Autenticação

- Basic
- Digest
- OAuth2
- Token Based
- JWT

Autenticação - Código de Status

- 401 Not Authorized (Não Autorizado)
- 403 Forbidden (Proibido)



Outros Pontos importantes a considerar
em sua API



Cache

- Ações não deve ser colocadas em cache (POST, PUT, DELETE, PATCH).
- Solicitações GET podem.
- Realizar o controle no lado do servidor com headers:
 - Last-Modified
 - ETag
- Caso informação não precise ser atualizada, informar:
 - 304, Not modified

Versionamento

URL:

<http://api.example.net/v2/cantores/michaeljackson>

<http://api.example.net/v2/albums/thisisit>

Content-type:

Accept: application/vnd.music.v2+json

Limites

- Permitir que o resultado de consultas sejam customizadas.
 - Utilizar Paginação de dados.
 - Permitir uma maneira de especificar intervalo de resultados (offset, limit).
 - Permitir especificar quais dados devem retornar (campos).
- Limitar taxa de acesso
 - Quantidade de acesso por período de tempo por usuário.
 - Regras diferentes para usuários pagos.

Tratamento de Erros

- Formato de saída de acordo com a expectativa do usuário
 - JSON -> JSON
 - XML -> XML
 - HTML -> HTML
 - etc..
- Utilize mensagens de erro significativas.
 - falta informar parâmetro? formato usado é valido ? função chamada não existe ? falta privilégios para efetuar a requisição ? etc..

Documentação

“Boas APIs possuem boa Documentação”

Documentação

- Documentação interativa

<http://apidocjs.com/>

<http://swagger.io/>

<http://www.mashery.com/api/io-docs>

Ferramentas de Apoio

- apigility (<https://apigility.org/>)
- Amazon API Gateway (<https://aws.amazon.com/pt/api-gateway/>)
- swagger (<http://swagger.io/>)

Referências

<http://www.novatec.com.br/livros/webservices-php/>

<http://restcookbook.com/>

<http://www.restapitutorial.com/httpstatuscodes.html>

<http://martinfowler.com/articles/richardsonMaturityModel.html>

<http://www.douglaspasqua.com/category/rest/>

